

# QuantumShield — CBOM Field Reference (for `cbom_writer.py`)

**Source standards:** CycloneDX 1.6+ (official CBOM) + original IBM CBOM extension (now upstreamed).

**Primary references:**

- <https://github.com/IBM/CBOM>
- [https://cyclonedx.org/guides/OWASP\\_CycloneDX-Authoritative-Guide-to-CBOM-en.pdf](https://cyclonedx.org/guides/OWASP_CycloneDX-Authoritative-Guide-to-CBOM-en.pdf)
- CycloneDX schema (bom-1.6 / 1.7)

This document is the **authoritative mapping** that Person B's `cbom_writer.py` must follow.

All generated components that represent detected crypto **must** be of type `cryptographic-asset` (CycloneDX) / historically `crypto-asset` (IBM).

---

## Core Component Fields

| Field                         | What it means                                                                  | Required?            | Example value for QuantumShield scanner                                         |
|-------------------------------|--------------------------------------------------------------------------------|----------------------|---------------------------------------------------------------------------------|
| <code>type</code>             | Component type.<br>Must be <code>"cryptographic-asset"</code> (CycloneDX 1.6+) | Yes                  | <code>"cryptographic-asset"</code>                                              |
| <code>name</code>             | Human-readable name of the asset                                               | Yes                  | <code>"RSA-2048"</code> ,<br><code>"AES-128-GCM"</code> ,<br><code>"MD5"</code> |
| <code>bom-ref</code>          | Unique identifier inside the BOM (used by dependencies)                        | Strongly recommended | <code>"crypto/algorithm/rsa-2048@&lt;hash-or-uuid&gt;"</code>                   |
| <code>cryptoProperties</code> |                                                                                | Yes                  | see below                                                                       |

| Field | What it means                              | Required? | Example value for QuantumShield scanner |
|-------|--------------------------------------------|-----------|-----------------------------------------|
|       | Container for all crypto-specific metadata |           |                                         |

## cryptoProperties Object

| Field                                        | What it means                                                   | Required?   | Example                                    |
|----------------------------------------------|-----------------------------------------------------------------|-------------|--------------------------------------------|
| <code>assetType</code>                       | One of the four CBOM categories                                 | Yes         | <code>"algorithm"</code> (common for our s |
| <code>algorithmProperties</code>             | Present when <code>assetType = "algorithm"</code>               | Conditional | see next table                             |
| <code>certificateProperties</code>           | Present when <code>assetType = "certificate"</code>             | Conditional | (out of scope for scanner)                 |
| <code>relatedCryptoMaterialProperties</code> | Present when <code>assetType = "related-crypto-material"</code> | Conditional | (keys, secrets —                           |
| <code>protocolProperties</code>              | Present when <code>assetType = "protocol"</code>                | Conditional | (TLS etc. — futur                          |
| <code>oid</code>                             | Object Identifier if known                                      | Optional    | <code>"1.2.840.1133</code><br>(RSA)        |
| <code>classicalSecurityLevel</code>          | Classical bit strength                                          | Optional    | <code>128</code> , <code>256</code>        |
| <code>nistQuantumSecurityLevel</code>        | NIST PQC security category (0–6)                                | Optional    | <code>0</code> for classical-or            |

## **algorithmProperties (when assetType = algorithm)**

| Field                                         | What it means                     | Example values                                                |
|-----------------------------------------------|-----------------------------------|---------------------------------------------------------------|
| <code>primitive</code>                        | Cryptographic building block      | "signature", "pke", "blockcipher" / "ae", "hash", "key-agree" |
| <code>variant / parameterSetIdentifier</code> | Specific instantiation            | "RSA-2048", "AES-128-GCM", "256"                              |
| <code>mode</code>                             | Mode of operation (block ciphers) | "gcm", "cbc", "ecb"                                           |
| <code>padding</code>                          | Padding scheme if relevant        | "oaep", "pkcs1", "none"                                       |
| <code>cryptoFunctions</code>                  | Operations observed or supported  | ["encrypt", "decrypt"], ["sign", "verify"], ["digest"]        |
| <code>executionEnvironment</code>             | Where it runs                     | "software-plain-ram" (default for source scan)                |
| <code>implementationPlatform</code>           | CPU / platform if known           | "x86_64", omit if unknown                                     |
| <code>certificationLevel</code>               | FIPS etc.                         | ["none"] or omit                                              |

### **Recommended mapping for QuantumShield v1 detections:**

| Detected keyword / family | primitive         | variant / parameterSetIdentifier | notes                           |
|---------------------------|-------------------|----------------------------------|---------------------------------|
| RSA                       | pke or signature  | RSA-2048 (or detected key size)  | Default to 2048 if size unknown |
| ECDSA                     | signature         | ECDSA + curve if known           |                                 |
| DH / ECDH                 | key-agree         | DH / ECDH                        |                                 |
| AES                       | ae or blockcipher | AES-128-GCM / AES-256            | Prefer mode if visible          |

| Detected keyword / family | primitive | variant / parameterSetIdentifier | notes        |
|---------------------------|-----------|----------------------------------|--------------|
| MD5                       | hash      | MD5                              | Mark as weak |

## Detection Context & Evidence (Critical for scanner)

CycloneDX attaches location evidence under the component's `evidence` object (preferred modern form).

IBM CBOM originally used a dedicated `detectionContext` array; both styles are accepted by tooling, but **prefer the standard `evidence.occurrences` form.**

| Field path                                            | What it means                                        | Example                                                            |
|-------------------------------------------------------|------------------------------------------------------|--------------------------------------------------------------------|
| <code>evidence.occurrences[].location</code>          | File path where the match occurred                   | <code>"src/auth.py"</code>                                         |
| <code>evidence.occurrences[].line</code>              | Line number (integer)                                | <code>42</code>                                                    |
| <code>evidence.occurrences[].additionalContext</code> | Optional snippet or keyword that triggered detection | <code>"RSA.generate"</code> or <code>"MessageDigest.getIns"</code> |
| confidence (0.0–1.0)                                  | How sure the detector is                             | <code>0.85</code> for pure regex, higher                           |

### QuantumShield v1 rule:

- Always populate at least one occurrence with `location` + `line`.
- Set confidence  $\approx 0.7$ – $0.9$  for regex matches (lower if the match is only a string constant).

---

## Dependencies: `implements` vs `uses`

CBOM explicitly distinguishes two dependency semantics:

| dependencyType          | Meaning                                                                                              | When to use                            |
|-------------------------|------------------------------------------------------------------------------------------------------|----------------------------------------|
| <code>implements</code> | A library / component <b>provides</b> the algorithm (i.e. the code that contains the implementation) | Scanning a crypto library itself       |
| <code>uses</code>       | Application code <b>calls</b> / <b>consumes</b> the algorithm                                        | Scanning application source (our case) |

### QuantumShield scanner policy (locked):

Because we scan application / sample source code, **default every dependency edge from the scanned component to a crypto-asset to `uses`**.

Do **not** emit `implements` unless the scanned file is clearly part of a crypto library implementation.

In CycloneDX 1.6+ the relationship is expressed via the top-level `dependencies` array (ref  $\rightarrow$  dependsOn) plus optional richer semantics; for the PoC it is sufficient to list the crypto-assets and record the evidence locations. Full dependency graph enrichment can come later.

---

## Minimal Valid Component Skeleton (copy-paste for writer)

```
{
  "type": "cryptographic-asset",
  "name": "RSA-2048",
  "bom-ref": "crypto/algorithm/rsa-2048@example",
  "cryptoProperties": {
```

```
    "assetType": "algorithm",
    "algorithmProperties": {
      "primitive": "pke",
      "parameterSetIdentifier": "2048",
      "cryptoFunctions": ["keygen", "encrypt", "decrypt"]
    }
  },
  "evidence": {
    "occurrences": [
      {
        "location": "src/crypto_utils.py",
        "line": 17,
        "additionalContext": "RSA"
      }
    ]
  }
}
```

---

## Notes for Person B

1. Always set `bomFormat: "CycloneDX"` and `specVersion: "1.6"` (or `"1.7"`).
2. Put the scanned project itself under `metadata.component` (type `application` or `library`).
3. Every crypto finding becomes a separate component of type `cryptographic-asset`.
4. Prefer `evidence.occurrences` over any custom `detectionContext` field for maximum tool compatibility.
5. Confidence is optional but useful; start with a constant 0.8 for regex hits.
6. Do **not** invent OIDs; only include them when you have a reliable mapping.

This reference is intentionally concise. Expand only when the scanner starts emitting certificates, keys or protocols.